天池新闻竞赛

方案：零基础入门推荐系统-新闻推荐 正式赛 第二名思路分享

1. 数据处理

train_click_log.csv

user_id	click_article_id	click_timestamp	click_environment	click_deviceGroup	click_os	click_country	click_regior
199999	160417	1.51E+12	4	1	17	1	
199999	5408	1.51E+12	4	1	17	1	
199998	96613	1.51E+12	4	1	17	1	
199998	144978	1.51E+12	4	1	17	1	
199998	285371	1.51E+12	4	1	17	1	
199997	224171	1.51E+12	4	1	17	1	

为了构造稳定的线下验证，从训练集用户中随机采样5w个用户作为作为线下验证集用户，将验证集用户的最后一次点击记录从原训练集的点击日志中剔除。合并这时候的训练集点击日志和测试集点击日志作为总的历史点击记录，预测验证集用户的最后一次点击作为线下验证，实验证明该验证方式很稳定，和线上指标同增同减且相差不大。

2. 多路召回

2.1 ItemCF

2.1.1 用户历史点击序列

按照 user_id 分组，聚合每个用户点击的文章 ID 列表，生成一个字典 user_item_dict，键为用户 ID，值为该用户点击文章列表。

代码块

```
user_item_ = df.groupby('user_id')['click_article_id'].agg(lambda x: list(x)).reset_index()
user_item_dict = dict(zip(user_item_['user_id'], user_item_['click_article_id']))
```

小红书
小红书号:9362252433

2.1.2 Item相似度矩阵

- **局部加权累加**

对于同一用户的点击序列中，每对不同文章 i 和 j（其中 i 出现在位置 loc1，j 出现在位置 loc2），先计算一个局部贡献值：

$$\text{weight} = \frac{\alpha \times 0.9^{(|loc2-loc1|-1)}}{\log(1+N)}$$

  - α 是位置信息权重，取值为 1.0 如果 loc2 > loc1（即正向点击顺序），否则为 0.7（反向点击顺序）。
  - N 是该用户点击的文章总数，即 $\text{len}(\text{items})$。
  - $0.9^{(|loc2-loc1|-1)}$ 表示位置差的指数衰减。

对于每个用户，将所有符合条件的文章对的贡献值累加，得到一个累积相似度 c_{ij}。

- **归一化**

$$\text{sim}(i,j) = \frac{c_{ij}}{\sqrt{\text{count}(i) \times \text{count}(j)}}$$

  - $\text{count}(i)$ 是文章 i 在所有用户点击记录中出现的总次数。

最终，相似度 $\text{sim}(i,j)$ 就是所有用户中该局部贡献的加权和除以各自点击频次的归一化因子。

2.1.3 召回

对于每个查询（df_query 中的每一行包含 user_id 和 item_id）：

- 通过获取 df_query 中所有用户的唯一 ID，并随机打乱顺序。根据最大线程数（n_split）将所有用户均匀分为若干批次。在开始召回前，清空临时文件夹，避免历史文件干扰。
- 对每个批次调用并行的 recall 函数，传入当前批次的用户查询、相似度字典和用户点击字典，并用当前批次的起始索引作为 worker_id。
  - 检查用户是否存在点击记录（若不存在则跳过）。
  - 对于该用户，将其历史点击序列倒序取最近两次点击（只保留最后两次点击）。针对每个历史点击文章，查询该文章在相似度字典 item_sim 中与其他文章的相似度，取前200个相似文章（排序后截取）。
  - 若候选文章不在用户的历史点击中，则累加其相似度分数，且对不同历史点击位置加权（乘以 0.7**loc，位置越靠前权重越高），倒排截断100。
  - 最后对每个用户生成候选文章的 DataFrame，记录用户 ID、候选文章 ID、相似度得分以及 label（如果 item_id 为 -1，则 label 为 NaN，否则将实际点击的文章置为 1，其余置为 0）。
- 使用 multitasking.wait_for_tasks() 等待所有并行召回任务完成。遍历临时文件夹，将每个批次生成的 pickle 文件加载后合并成一个总的 DataFrame df_data。

小红书
小红书号:9362252433

- 对结果按照 user_id 和 sim_score（降序）排序，确保每个用户的召回候选按照得分排序。

2.2 基于网络关系的召回()

2.2.1 文章的点击用户列表

代码块

```
1       item_user_ = df.groupby('click_article_id')
    ['user_id'].agg(list).reset_index()
2       item_user_dict = dict(zip(item_user_['click_article_id'],
    item_user_['user_id']))
```

2.2.2 Item相似度矩阵

对于每个用户的点击记录，通过共现用户的数量来计算相似度，使用如下公式：

$$\text{sim}(i,j) += \frac{1}{\log(|U_i|+1) \times \log(|U_j|+1)}$$

其中，$|U_i|$ 和 $|U_j|$ 分别是文章 i 和 j 被用户点击的数量。

2.2.3 召回

- 遍历查询数据 df_query 中每个用户的查询（包括 user_id 和 item_id）。如果用户在 user_item_dict 中有点击记录，提取用户的历史点击文章（只取最后一次点击）。
- 对于每篇历史点击文章，从预先计算的相似度信息中取前100个相似文章。
- 累加倒排截断50
- 对于每个用户，生成一个候选文章的 DataFrame，记录 user_id、article_id、sim_score 和 label（若 item_id 为 -1，则 label 为 NaN，否则为 1，表示真实点击）。
- 最后将召回结果保存为 pickle 文件。

2.3 w2v 向量召回

2.3.1 Word2Vec 模型训练与文章向量构建

- **数据准备**：将输入 DataFrame 按照指定字段（例如 ‘user_id’ 和 ‘click_article_id’）进行分组，聚合成列表，形成“句子”集合，每个句子代表一个用户的点击序列。
- **预处理**：将每个文章 ID 转为字符串，便于 Word2Vec 处理，并汇总所有词（文章 ID）。
- **模型加载或训练**：如果指定路径下已存在模型文件，则直接加载；否则用 gensim 的 Word2Vec 进行训练，参数设置包括向量维度 256、窗口大小 3、使用 Skip-gram 模型（sg=1）、负采样（negative=5）等。

小红书
小红书号:9362252433

```
代码块
        model = Word2Vec(sentences=sentences,
                         size=256,
                         window=3,
                         min_count=1,
                         sg=1,
                         hs=0,
                         seed=seed,
                         negative=5,
                         workers=10,
                         iter=1)
        model.save(f'{model_path}/w2v.m')
```

- **构建文章向量映射**：遍历所有出现过的文章，将模型中对应的向量取出，转换成整数型键（文章 ID）和对应的向量值，生成字典 article_vec_map。

```
代码块
    for word in set(words):
        if word in model:
            article_vec_map[int(word)] = model[word]
```

2.3.2 召回

- **遍历查询数据**：对于每个查询记录（包含用户 ID 和目标 item_id），从全局的 user_item_dict 中获取该用户的点击历史，并取最后一次点击（[-1:]）。
- **利用 Annoy 索引查询相似文章**：
  - 对于用户最近点击的文章，从 article_vec_map 获取对应向量，并利用 Annoy 索引查找最近邻的 100 个文章（同时返回距离）。
  - 将距离转换成相似度得分（公式为 sim_score = 2 - distance，距离越小，相似度越高）。
- **候选文章评分**：对于每个候选文章，如果不在用户历史点击中，则累加其相似度分数（累加方式直接求和）。最终对每个用户排序，取得分最高的 50 个文章作为召回候选。
- **构造召回结果 DataFrame**：为每个用户构造包含 user_id、article_id、sim_score 以及标签（label：如果目标 item_id 不为 -1，则将目标文章标记为 1，其余为 0；若为 -1，则 label 为 NaN）的 DataFrame，并保存到临时文件中，文件名由 worker_id 命名。

2.4 多路召回合并

2.4.1 相似度分数归一化

```
代码块
def mms(df):
```

小红书号:9362252433

```
    user_score_max = {}
    user_score_min = {}

    # 获取用户下的相似度的最大值和最小值
    for user_id, g in df[['user_id', 'sim_score']].groupby('user_id'):
        scores = g['sim_score'].values.tolist()
        user_score_max[user_id] = scores[0]
        user_score_min[user_id] = scores[-1]

    ans = []
    for user_id, sim_score in tqdm(df[['user_id', 'sim_score']].values):
        ans.append((sim_score - user_score_min[user_id]) /
                   (user_score_max[user_id] - user_score_min[user_id]) +
                   10**-3)
    return ans
```

2.4.2 合并召回结果

- 为每种方法设置权重：itemcf 与 binetwork 的权重为 1， w2v 的权重为 0.1。
- 使用 pd.concat 将各方法的召回结果拼接成一个总的 DataFrame。
- 按照 user_id 与 article_id 分组，对 sim_score 进行求和，生成每个用户-文章对的综合得分。
- 对原始召回结果进行去重（保留 user_id、article_id 以及 label 信息），然后与综合得分合并。
- 根据 user_id 升序、sim_score 降序对最终结果进行排序。

2.4.3 筛选“有用”的召回结果

- 如果某用户召回结果中包含 NaN 标签（表示测试用户），则保留该组；
- 否则统计该用户所有召回记录中的正样本标签（label == 1），只保留正样本数为 1 的用户记录，正样本数大于 1 时输出错误信息。
- 合并筛选后的用户召回结果为 df_useful_recall 并重新排序。

3. 特征工程

3.1 用户-新闻交互特征

- 用户历史点击文章与待预测文章的相似度总和
  - 对每篇历史点击文章与待预测文章计算相似度，并乘以位置衰减因子 0.7**loc，累加相似度得分。
- 用户最后一次点击文章与待预测文章的相似度
- 用户最后一次点击文章与待预测文章在二部网络（binetwork）中的相似度

小红书
小红书号:9362252433

- 用户历史点击的文章与待预测文章之间的余弦相似度总和
- 用户最后一次点击文章与待预测文章之间的余弦相似度：

3.2 用户行为特征

- 点击文章的创建时间差的平均值
- 用户点击文章时间差的平均值
- 点击文章的创建时间差的统计值
- 点击文章的小时分布（click_datetime_hour）；
- 文章的 words_count（字数）统计特征；
- 文章的 created_at_ts 和 click_timestamp 相关的统计特征。

3.3 新闻特征

4. 排序

4.1 LightGBM 分类模型 (LGBMClassifier)

- num_leaves: 树的最大叶子数；
- max_depth: 最大树深；
- learning_rate: 学习率；
- n_estimators: 迭代次数；
- subsample, feature_fraction: 数据和特征的采样比例；
- reg_alpha, reg_lambda: 正则化项；
- importance_type: 特征重要性计算方法。

代码块

```
df_train = df_feature[df_feature['label'].notnull()]
df_test = df_feature[df_feature['label'].isnull()]

del df_feature
gc.collect()

ycol = 'label'
feature_names = list(
    filter(
        lambda x: x not in [ycol, 'created_at_datetime', 'click_datetime'],
```

小红书号:9362252433

```
            df_train.columns))
    feature_names.sort()

    model = lgb.LGBMClassifier(num_leaves=64,
                               max_depth=10,
                               learning_rate=0.05,
                               n_estimators=10000,
                               subsample=0.8,
                               feature_fraction=0.8,
                               reg_alpha=0.5,
                               reg_lambda=0.5,
                               random_state=seed,
                               importance_type='gain',
                               metric=None)

    oof = []
    prediction = df_test[['user_id', 'article_id']]
    prediction['pred'] = 0
```

4.2 交叉验证与训练

代码块

```
    kfold = GroupKFold(n_splits=5)
    for fold_id, (trn_idx, val_idx) in enumerate(
            kfold.split(df_train[feature_names], df_train[ycol],
                        df_train['user_id'])):
        X_train = df_train.iloc[trn_idx][feature_names]
        Y_train = df_train.iloc[trn_idx][ycol]

        X_val = df_train.iloc[val_idx][feature_names]
        Y_val = df_train.iloc[val_idx][ycol]

        log.debug(
            f'\nFold_{fold_id + 1} Training ================================\n'
        )

        lgb_model = model.fit(X_train,
                              Y_train,
                              eval_names=['train', 'valid'],
                              eval_set=[(X_train, Y_train), (X_val, Y_val)],
                              verbose=100,
                              eval_metric='auc',
                              early_stopping_rounds=100)

        pred_val = lgb_model.predict_proba(
```

小红书
小红书号:9362252433

```
            X_val, num_iteration=lgb_model.best_iteration_)[:, 1]
        df_oof = df_train.iloc[val_idx][['user_id', 'article_id', ycol]].copy()
        df_oof['pred'] = pred_val
        oof.append(df_oof)

        pred_test = lgb_model.predict_proba(
            df_test[feature_names], num_iteration=lgb_model.best_iteration_)[:,
                                                                              1]
        prediction['pred'] += pred_test / 5

```

小红书
小红书号:9362252433